

PROJETO DE ENGENHARIA DE SOFTWARE

NSA Metrics Analyzer

É possível prever quais arquivos .java são propensos a risco de segurança a partir de métricas de qualidade de código?

MSR

ISO/IEC 25010

Machine Learning

Igor Mariano

202407095992

Filipe Oliveira

202303161085

Por que prever risco a partir de qualidade?



Vulnerabilidades custam caro

Falhas de segurança são caras de corrigir e perigosas quando chegam à produção.



Revisão manual não escala

Auditar um arquivo por um arquivo é inviável em projetos com milhares de arquivos.



Onde olhar primeiro?

Priorizar a inspeção nos arquivos mais arriscados poupa tempo e reduz risco.

Ideia central

Complexidade, duplicação e histórico de mudanças podem indicar quais arquivos inspecionar primeiro.

O que é o NSA Metrics Analyzer

Pipeline automatizado e reprodutível de mineração de repositórios de software

1

100% estático

Só o código-fonte clonado:
sem build e sem execução.

2

10 repositórios NSA

Projetos Java reais e públicos
usados no estudo

3

Dataset por arquivo

Uma linha por arquivo .java
com métricas consolidadas.

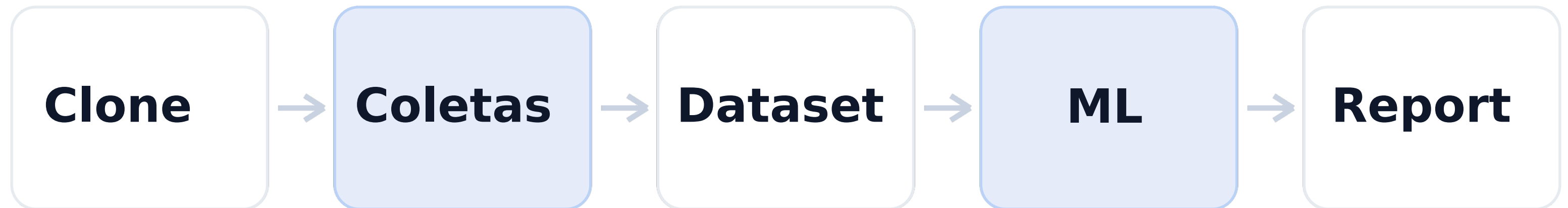
4

ISO/IEC 25010

Métricas organizadas por
dimensão de qualidade.



Fluxo geral do pipeline



Ferramentas de segurança e qualidade



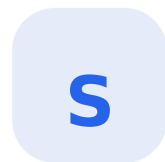
Semgrep
SAST

Define o alvo de segurança. A união com CodeQL marca arquivos com risco.



CodeQL
SAST

Complementa o Semgrep e amplia a cobertura de achados de segurança.



SonarQube
servidor Docker

Gera complexidade, duplicação, code smells e dívida técnica.



lizard
offline

Fornece complexidade ciclomática como fallback estrutural.

Ferramentas estruturais e históricas



CK

orientação a objetos

Coleta WMC, DIT, CBO, RFC, LCOM, TCC e outras métricas OO.



PyDriller

histórico git

Extraí commits, autores, churn e idade do arquivo.



OSV

API

Identifica CVEs em dependências declaradas.



Secrets

Gitleaks + detect-secrets

Detecta segredos expostos no código.

Tamanhos dos repositórios

ghidra ~ 23,1 mil arquivos



datawave ~ 6,1 mil arquivos



demais projetos menos de 200 cada



O dataset em números

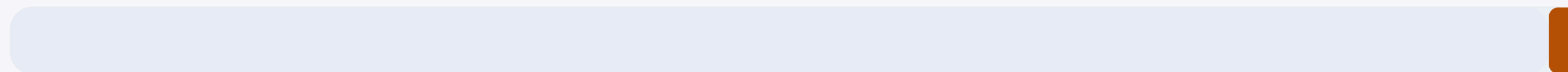
31.622
arquivos .java

10
repositórios

560
com risco

Classes fortemente desbalanceadas

1,77% positivos



sem risco: 31.062

com risco: 560

A métrica-alvo: `has_security_risk`

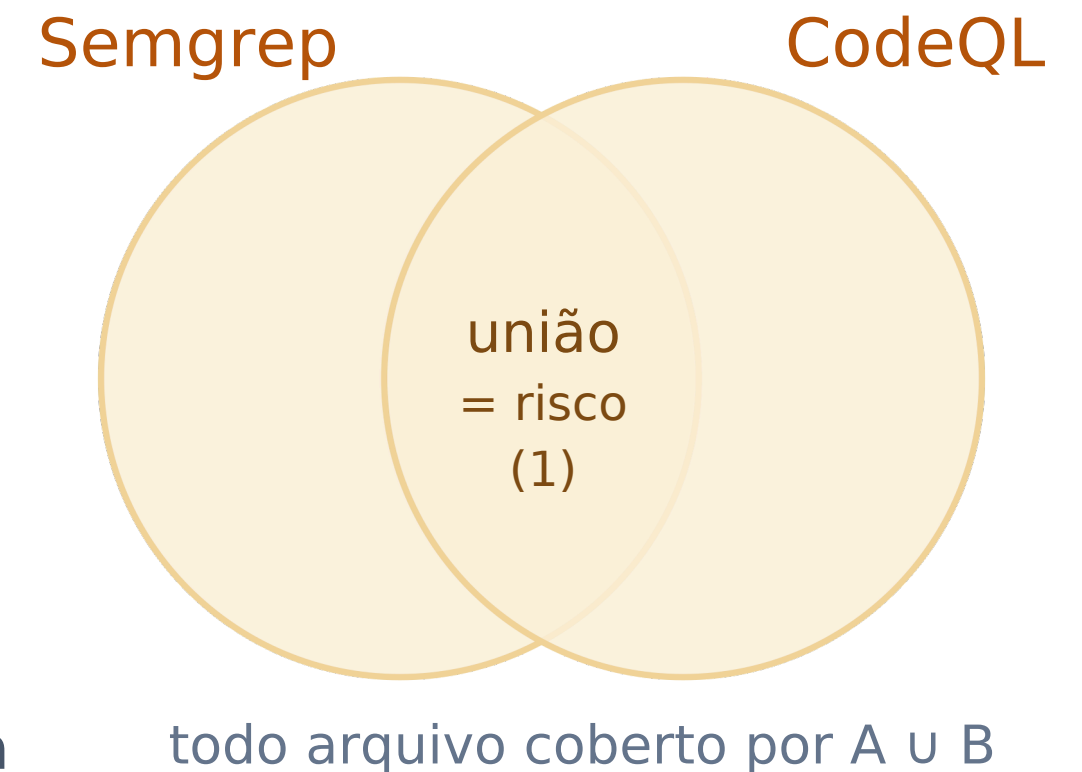
`has_security_risk` $\in \{ 0, 1 \}$ · binária

Vale 1 se o arquivo foi sinalizado pelo Semgrep OU pelo CodeQL - a união. Senão, 0

R Por que união, não interseção

Em segurança, o pior erro é o falso-negativo. A união maximiza o recall captura mais arquivos potencialmente arriscados.

Mapeia direto para a dimensão Segurança da ISO/IEC 25010.



Por que usar união e não interseção?

! **Prioridade: evitar falso-negativo**
Em segurança, o pior erro é deixar passar um arquivo realmente arriscado.

U **A união maximiza o recall**
Somar Semgrep e CodeQL captura mais arquivos potencialmente problemáticos.

n **A interseção seria conservadora demais**
Exigiria que as duas ferramentas concordassem, perdendo achados úteis para triagem.

Como separamos treino e teste

Sem split aleatório 70/30: validação cruzada agrupada por repositório (GroupKFold, k = 5).

	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Dobra 1	TESTE	TESTE								
Dobra 2			TESTE	TESTE						
Dobra 3					TESTE	TESTE				
Dobra 4							TESTE	TESTE		
Dobra 5									TESTE	TESTE

Treino

Teste (repositório reservado)



Evita vazamento intra-repositório

Arquivos do mesmo projeto se parecem; nunca ficam em treino e teste ao mesmo tempo.



Mede generalização real

Testar num projeto inteiro inédito simula o uso em um software novo. **k = 5 · cada repositório testado 1x**

Modelos e métricas de avaliação

5 MODELOS COMPARADOS

1	Regressão Logística	Linear · baseline
2	Árvore de Decisão	Árvore
3	Random Forest	Ensemble (bagging)
4	XGBoost	Boosting
5	LightGBM	Boosting

ROC-AUC por modelo



Conclusão

A hipótese se confirma

Métricas de qualidade têm poder preditivo sobre risco de segurança: ROC-AUC perto de 0,86

Limitações



Problema raro

Apenas 1,77% dos arquivos são positivos, isso reduz precision e F1.



Análise totalmente estática

O pipeline não compila, não executa testes e não observa comportamento em runtime.



Validação exigente

Testar em projetos inteiros inéditos é mais difícil, mas é mais honesto.

Trabalhos futuros

1

Ampliar a base

Adicionar mais repositórios Java com diferentes tamanhos afim de balancear a amostra dos dados

2

Incluir métricas dinâmicas

Usar execução, testes e comportamento real quando o build estiver disponível.

3

Ajustar o limiar

Escolher o ponto de decisão conforme o custo de falso-positivo e falso-negativo.

Obrigado!